



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Software design and implementation

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Famous claim

A portrait of Mark Andreessen, a man with a shaved head and a goatee, wearing a dark suit jacket over a light blue shirt. He is positioned in front of a golden, circular, halo-like background.

Mark Andreessen
founder of Netscape,
renowned Venture Capitalist
Andreessen-Horowitz

Software is eating the
world, in all sectors

In the future every
company will become a
software company

[August 2011]

Modern vision

Central and essential assumption of *modern software engineering* →

Design must consider product's entire life

An old and narrow perspective is based just on fast software development. Risks:

- Likely to lead to failures
- No reliability assurances
- Later expenses much higher than development costs



Software product's entire life

Validation

Testing

DESIGN

DEVELOPMENT

OPERATION

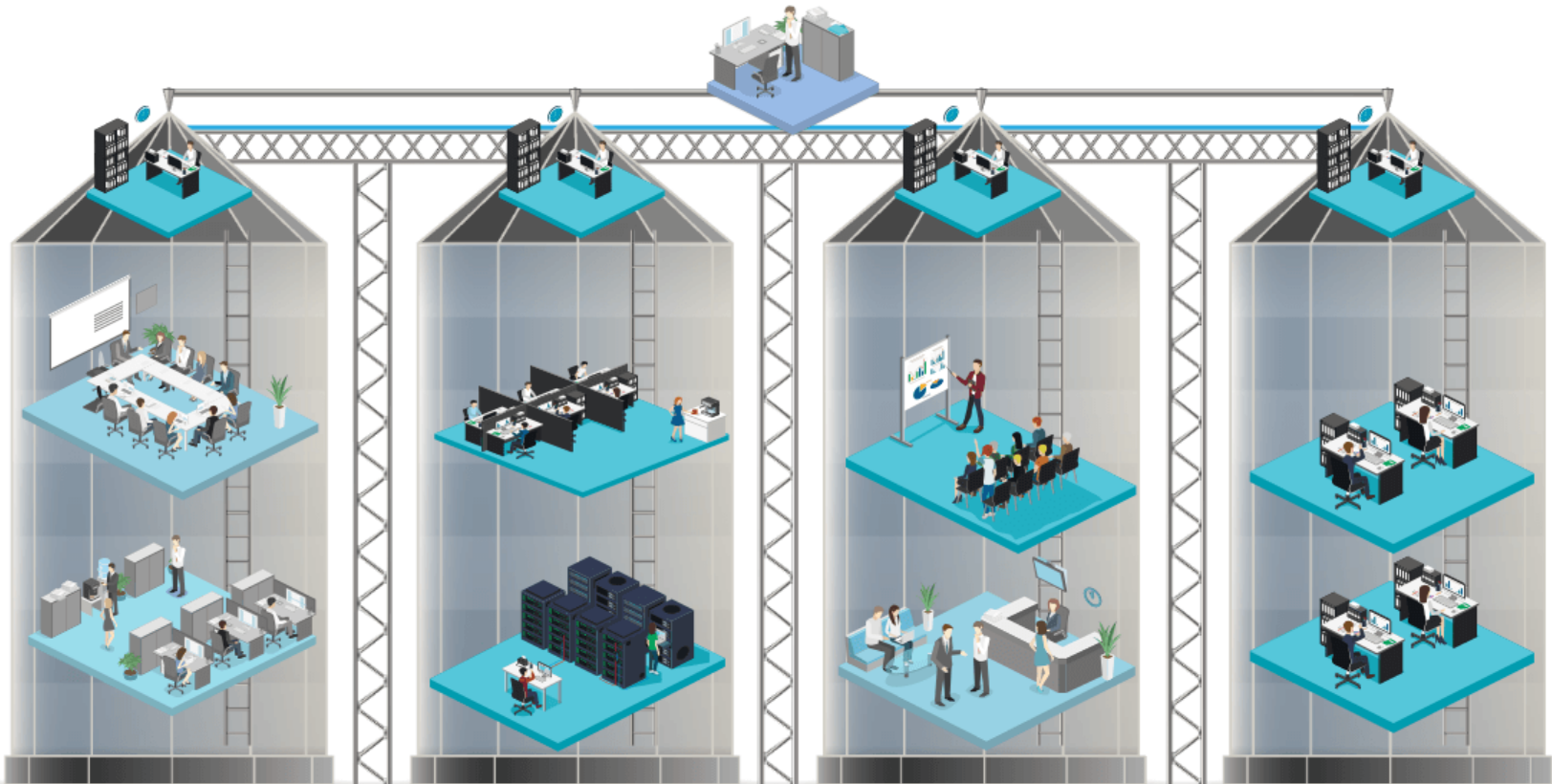
Check and continuous improvement



Organizational chart in a typical company



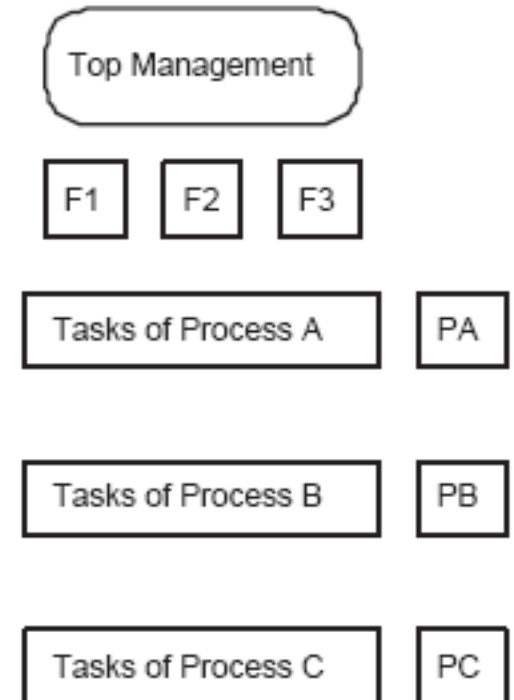
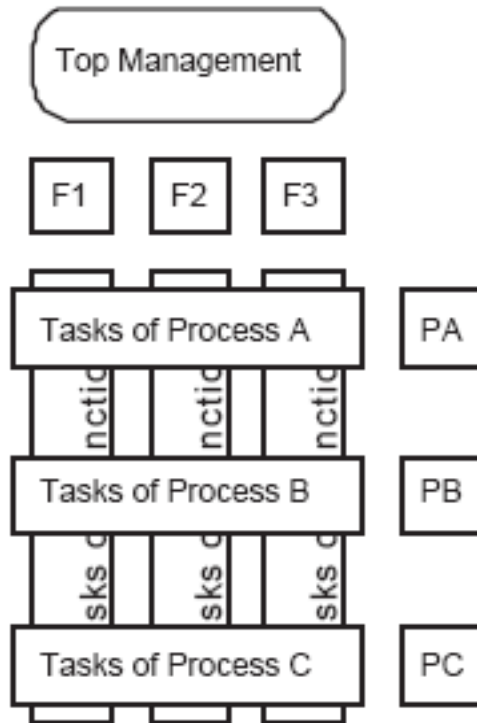
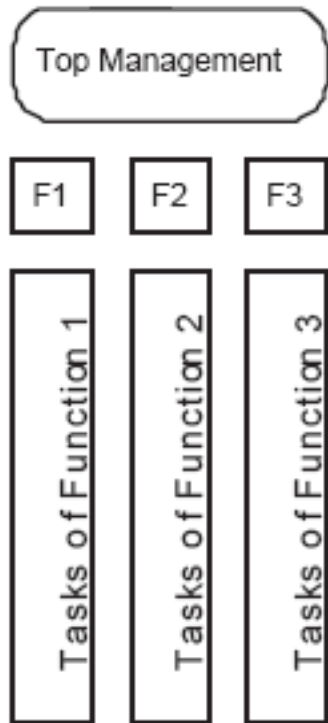
Silos reflect the organization of work (that bring to local optimizations)



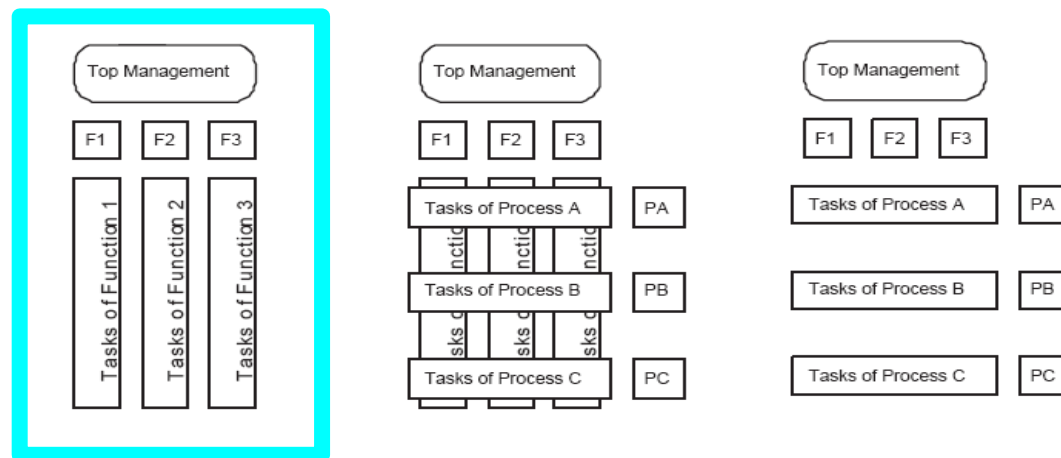
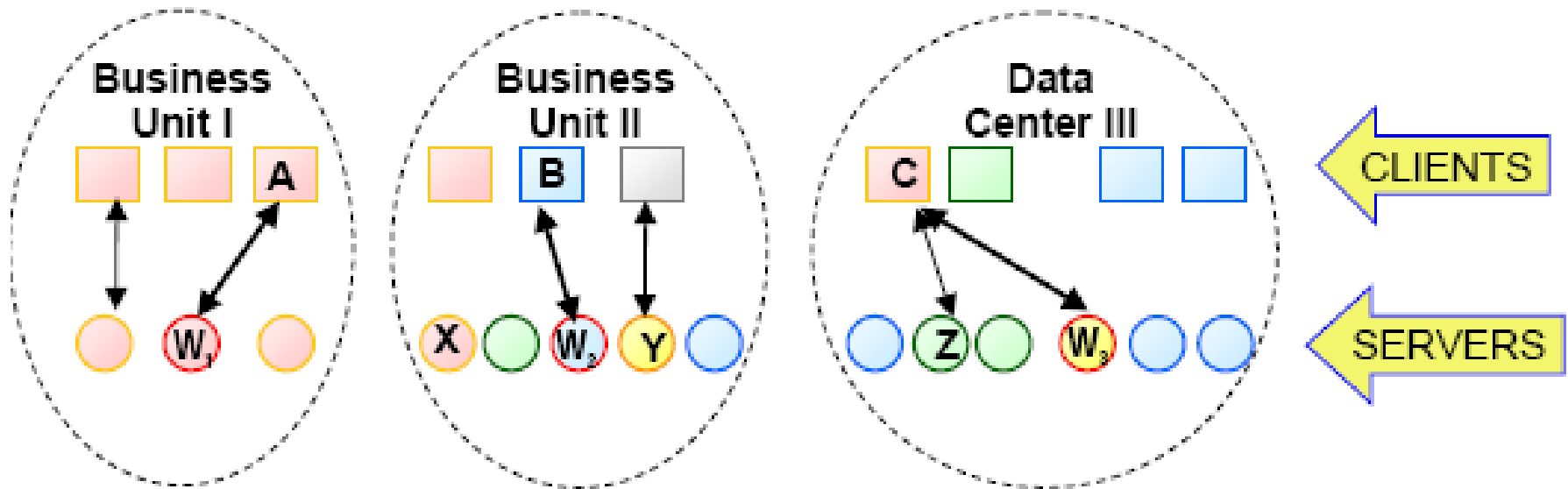
Acceptable in the old days when each department had its own mini-datacenter with related data and applications.
Today, cooperation among silos is unavoidable



Enterprise organization



Remote past: *Vertical silos*



Traditional silos (data and engineering)



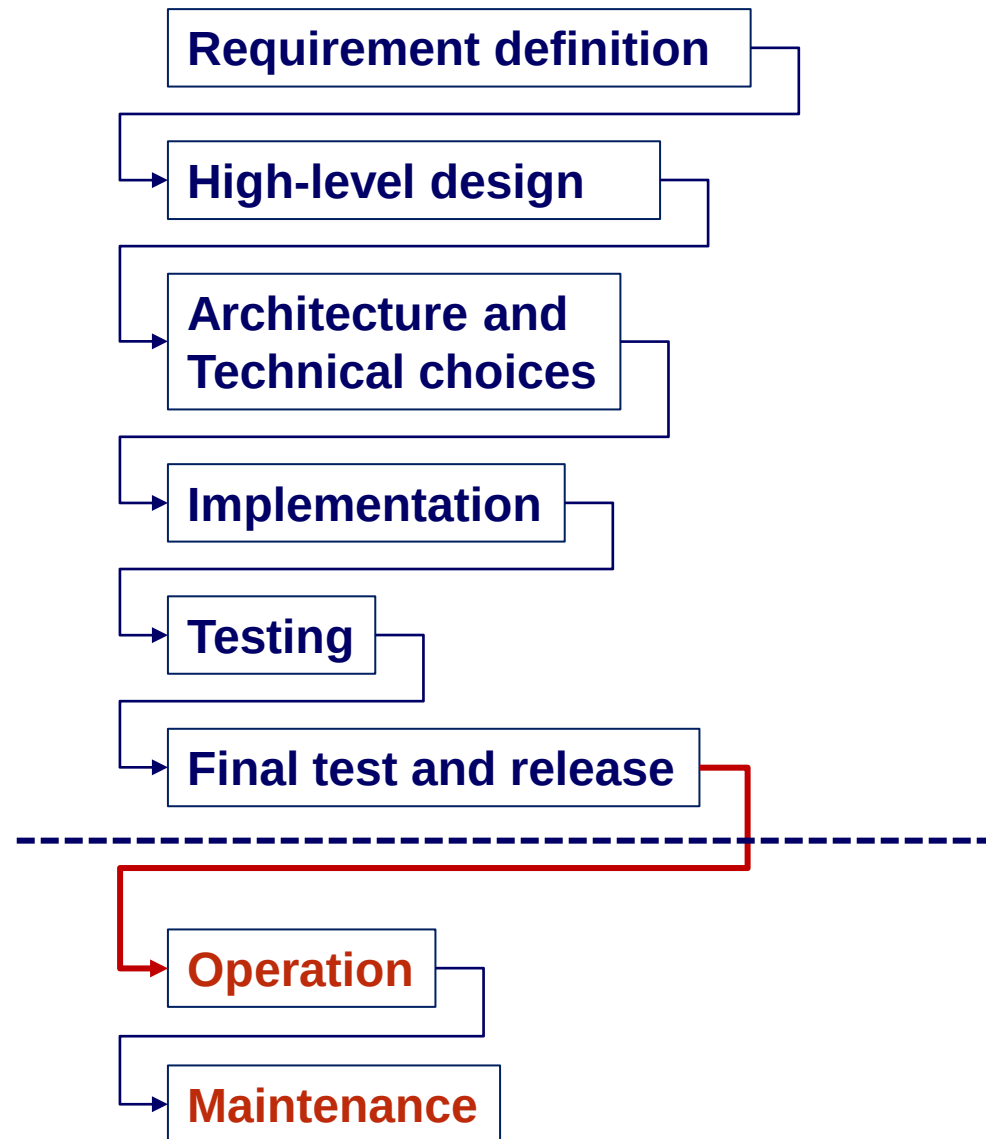
Silos reinforce the *waterfall model* of software production

- **Development**
 - Ship code
 - Get around blocks
 - Do minimal operations and security requirements
- **Operations**
 - Keep code stable
 - Block changes
 - Get involved because of other people code
- **Security**
 - Lock up the code
 - Block changes
 - Prescribe fixes
 - Work in a locked room
 - Obsessed with hats



Waterfall model

- It remains one fundamental abstraction model for the entire (system and software) development process
- It is a useful scheme even for comparison with more modern models

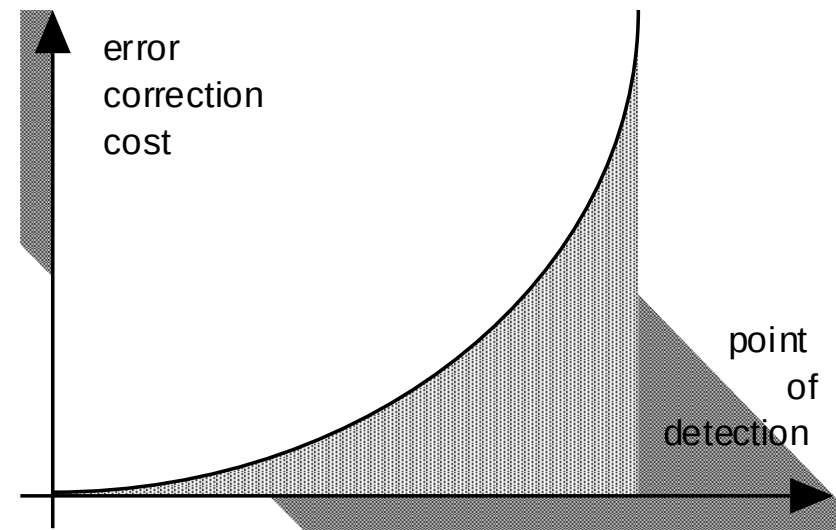


We know that ...

The cost of correcting an error grows very rapidly as the project moves along its life-cycle

This observation argues for early error detection and provides the motivation for technical reviews

The highest cost errors are those involving the systems requirements formulation



... but also that

- **Definition of requirements, project management, testing and operations are VERY difficult tasks**
- The theory is much ahead of the state of the practice:
 - Many standard frameworks, models, philosophic approaches, best practices
 - Difficulty to respect all specifications
 - In some cases, the complexity of documentation is similar to that of implementation → Inacceptable by most developers and marketing
 - *Time to market is critical* and few contexts accept to pay more for a quality product

**Books help, theory helps, but you can learn only through
practice, practice, practice**



Requirements

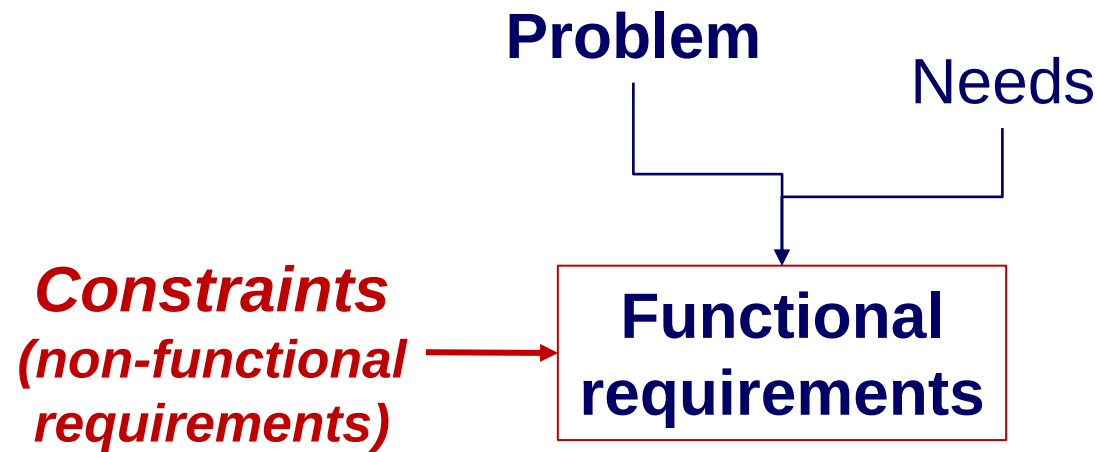


Requirement

- A **requirement** is a *technical* objective which is imposed upon the software, that is, anything that might affect the kind of software that is produced
- A requirement may be imposed by
 - the customer
 - the developer
 - the operating environment
- **The source, rationale, and nature of the requirement must be documented (even for contractual reasons)**
- Requirements fall into two broad categories
 - **Functional**
 - **Non-functional**



Requirements and issues



Functional requirements

- **Functional** requirements **are concerned with what the software must do**
 - capabilities, services, operations, input-processing-output, ...
- **Functional** requirements are **not concerned with how the software does things** → they must be free of design considerations



Other requirements

- Functional requirements are incomplete unless they capture all relevant aspects of the software's environment
 - they define the interactions between the software and the environment
 - the environment may consist of users, other systems, support hardware, operating system, etc.
 - the system/environment boundary must be defined



Non-functional requirements

Non-functional requirements restrict the range of all possible solutions to the acceptable ones

Non-functional requirements cover a broad range of issues

- performance constraints
- reliability constraint
- security constraint
- scalability objectives
- operating constraints
- life-cycle constraints (development and operation)
- economic constraints (development and operation)
- legal constraints
- manufacturing constraints



(Response) time constraint

- Time seems (and often is) a non-functional requirement
 - E.g., when running faster is “a good thing” but it does not change outcome
- However, some systems are inherently **time-dependent** and in such cases, time must be treated as a functional element:
 - human interaction, mechanical control, industrial control, etc.
 - making system run too much faster or slower may be harmful
- One technique is to transform time into events
 - A timer or other device has a specified setting (e.g., rate): T
 - The device generates events at times governed by that setting: $f(T)$
- Functional requirements can be written about T and $f(T)$
 - Per a model that represents the relationship between T and $f(T)$



Communication problems among internal and external teams

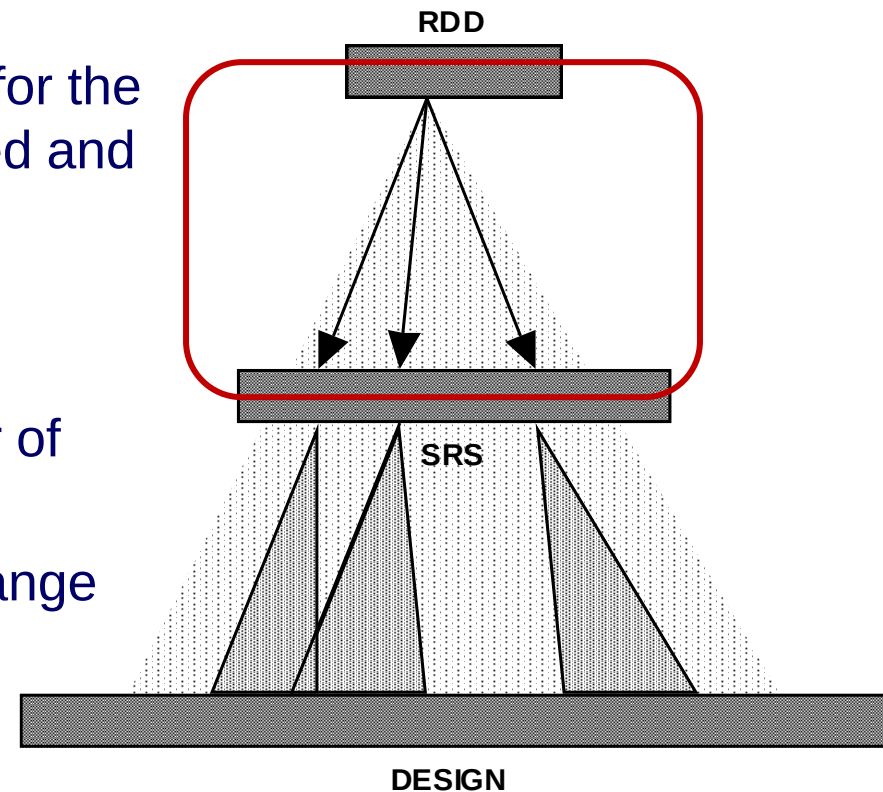
- Missing or vague requirements are a recipe for disaster
- Anything that is not made explicit may not be implemented or even considered
- Anything that is ambiguous is likely to get implemented in the least desirable way
- Standard practices may be omitted
- Cultural settings affect the way in which requirements are written and read
- Multiple views may be required to formulate a reasonably complete set of requirements for decisions



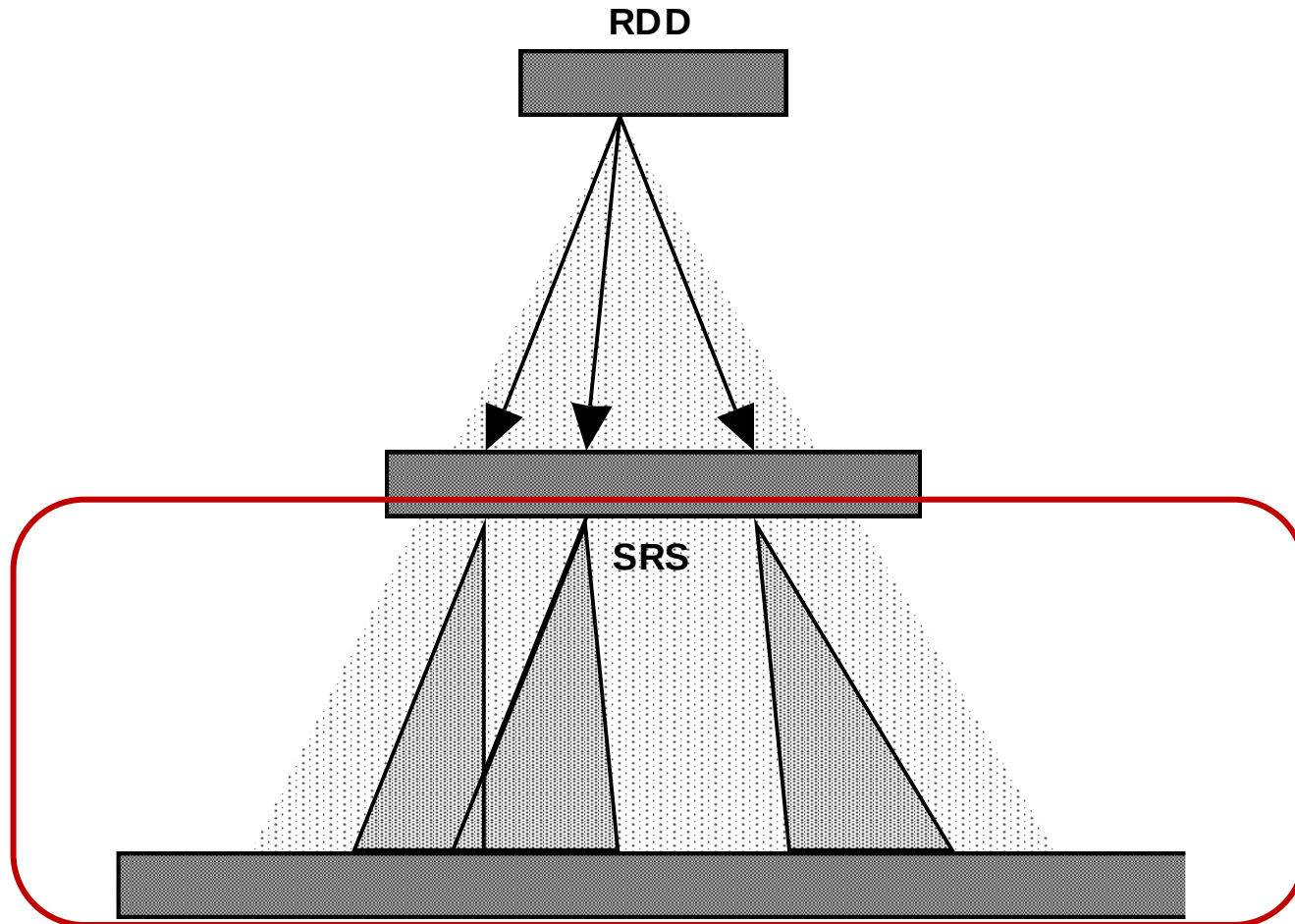
Requirements Definition Document (and then, Software Requirement Specification)

The Requirements Definition Document (RDD)

- is a document at a relatively high level of abstraction
- does not provide yet a baseline for the development (needs to be refined and extended during specification)
- does provide the basis for specification
- is the starting point for a number of specialized preliminary studies
- must be accessible to a broad range of readers: customers, users, designers, managers, etc.



Overall picture: RDD + SRS



ARCHITECTURE AND DESIGN

DETAILED DESIGN OF PROCESSES



Software Requirements Specification

- **Point of origin**
 - References to RDD
- **Purpose**
 - Providing a baseline for architecture design and all software development activities
- **Nature**
 - Technical
- **Focus**
 - Software/environment interactions
 - Technical reformulation of constraints
- **Usage for**
 - Software design
 - Testing activities



SRS: Table of Contents

(example IEEE STD-830-1984)

1. Introduction

2. General description (about the product perspectives and main functions)

3. Specific requirements (← *The most important Section*)

- External interface requirements (User, Hardware, Software, and Communication interfaces)
- Functional requirements (one sub-section for each requirement: description – input – processing – output – error handling)
- Use cases
- Non-functional requirements: performance, availability, security, maintainability, portability

4. Design constraints (constraints imposed by standards, company policies, limitations on HW/SW/OS choices)

5. Change management (what to do when something changes)

6. Appendices



SRS in practice

- The SRS document states in precise and explicit language **functions** and **capabilities** a software system (e.g., application, e-commerce site, social site) must provide, as well as states any required **constraints** by which the system must abide
- The SRS is often referred to as the “**parent document**” because all subsequent project management documents, such as design specifications, statements of work, software architecture specifications, testing and validation plans, and documentation plans, are related to SRS
- SRS contains functional and non-functional requirements only. It does not offer design suggestions, possible solutions to technology or business issues, or **any other information other than what the analysis team understands the customer’s system requirements to be**



The 4 main goals of SRS

- 1. It provides feedback to the customer.** An SRS is the customer's assurance that the development organization understands the issues or problems to be solved and the software behavior necessary to address those problems
- 2. It decomposes the problem into component parts**
- 3. It serves as an input to the design specification.** Therefore, the SRS must contain sufficient detail in the functional system requirements so that a design solution can be devised
- 4. It serves as a product validation check**



Hints for you

- SRS development is a demanding **collaborative effort**; hence you can learn a lot during this phase
- **If you are not a member of the SRS team, ask to be part of it** (even just for listening)
- It is likely that your company has developed SRS document before, so you can study and refer to:
 - **Company SRS template**
 - **Examples**



Requirements *verification*

- **Requirements verification is an activity directed towards the discovery of specification errors**
(this differs from *software verification* that is the process of checking that a software system meets specifications)
- The ultimate goal is to ensure that the specification documents (when considered on their own) are
 - correct
 - consistent
 - complete
- The verification must be carried out against a formal (e.g., definition language, state machine model, data-flow model), or informal model, possibly by third parties
 - Formal specifications can be checked out by tools



Requirements *validation*

- **Requirements validation aims to establish that specified requirements represent the needs of the customer and/or user**
- Needs are not reflected by any model or document
 - Thus, validation cannot be performed in a mechanical way
- Good communication is the key to a successful validation
 - well-defined terminology
 - well-written and simple specifications
 - formal reviews (by third party)
 - rapid prototypes
 - mock-ups
 - simulations



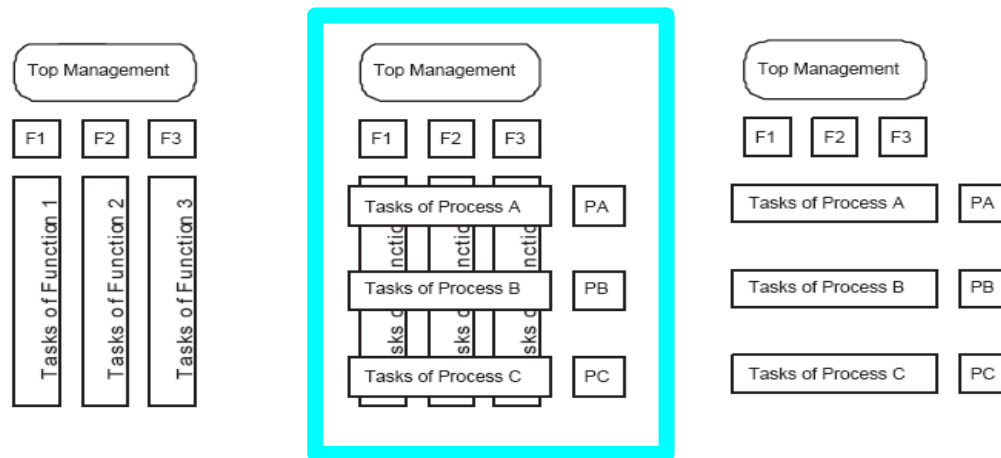
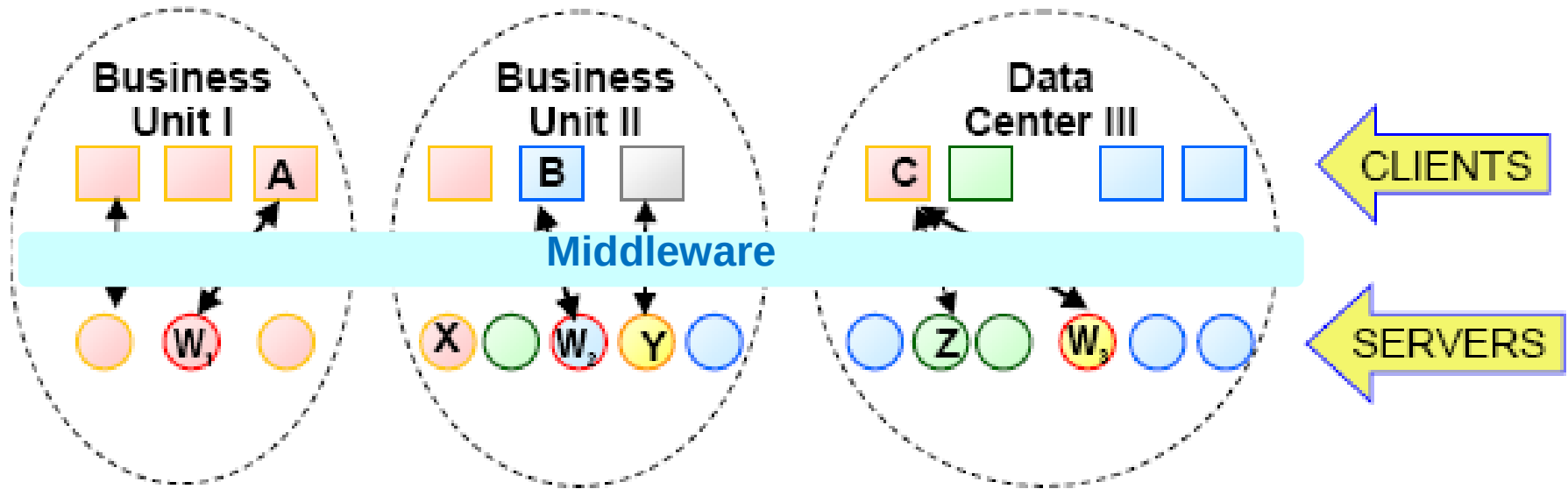
User Manual for validation

Problems

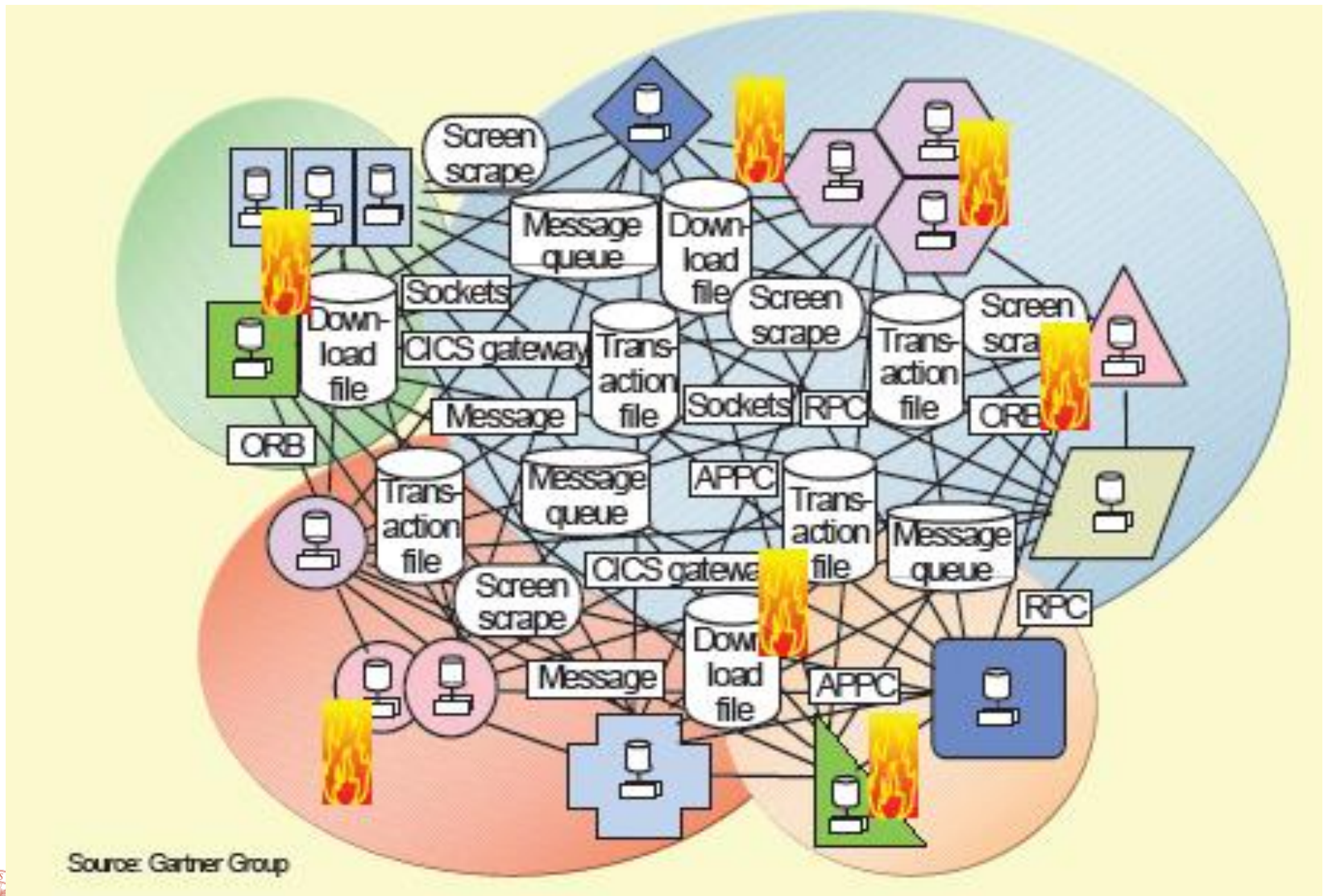
- Effective specifications often require the integration of multiple related models
- Human/computer interactions are too complex for commonly used requirements techniques
- For applications involving human interfaces, the User Manual can be used as a substitute for (and/or can grow out of) large sections of the SRS
 - Many companies have not even considered that this is the right time to write the User Manual
 - Other companies think that good products should not even use a User manual



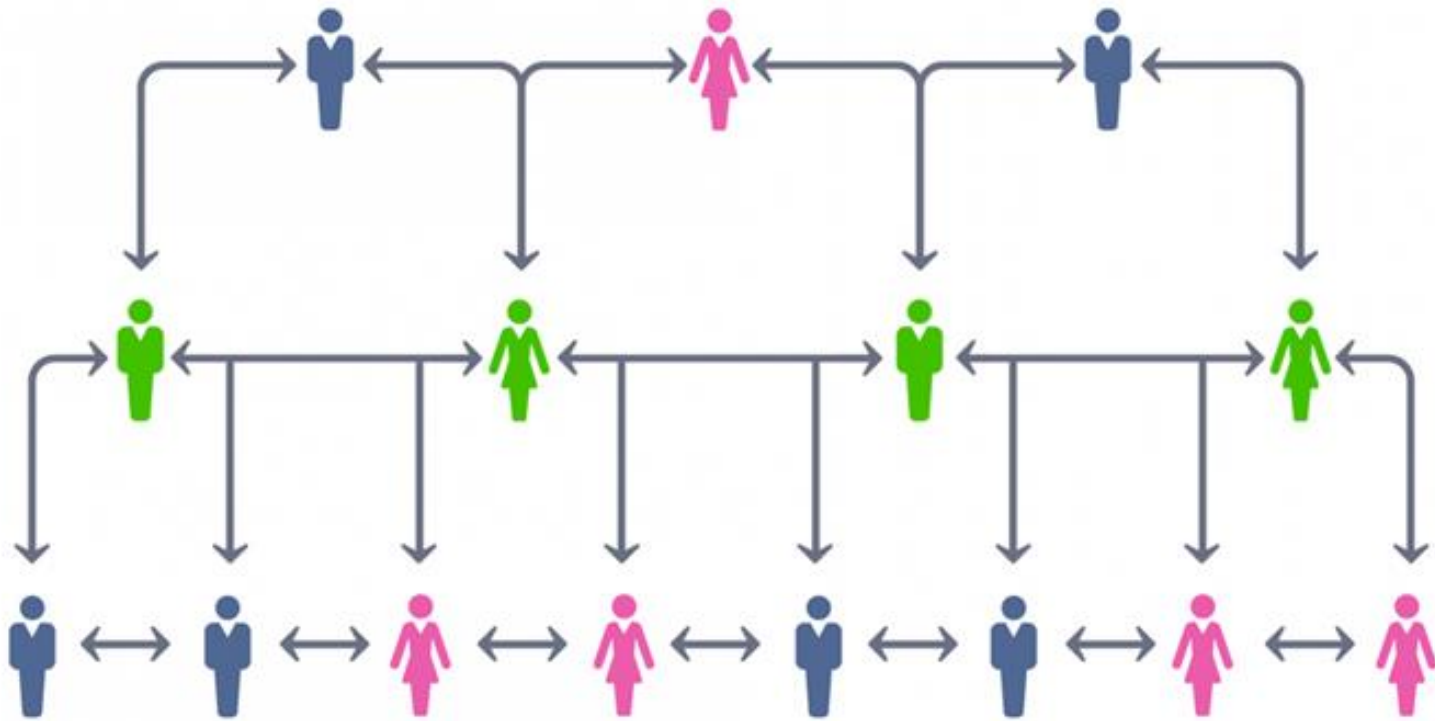
Integration



The typical consequence of trying to integrate silos



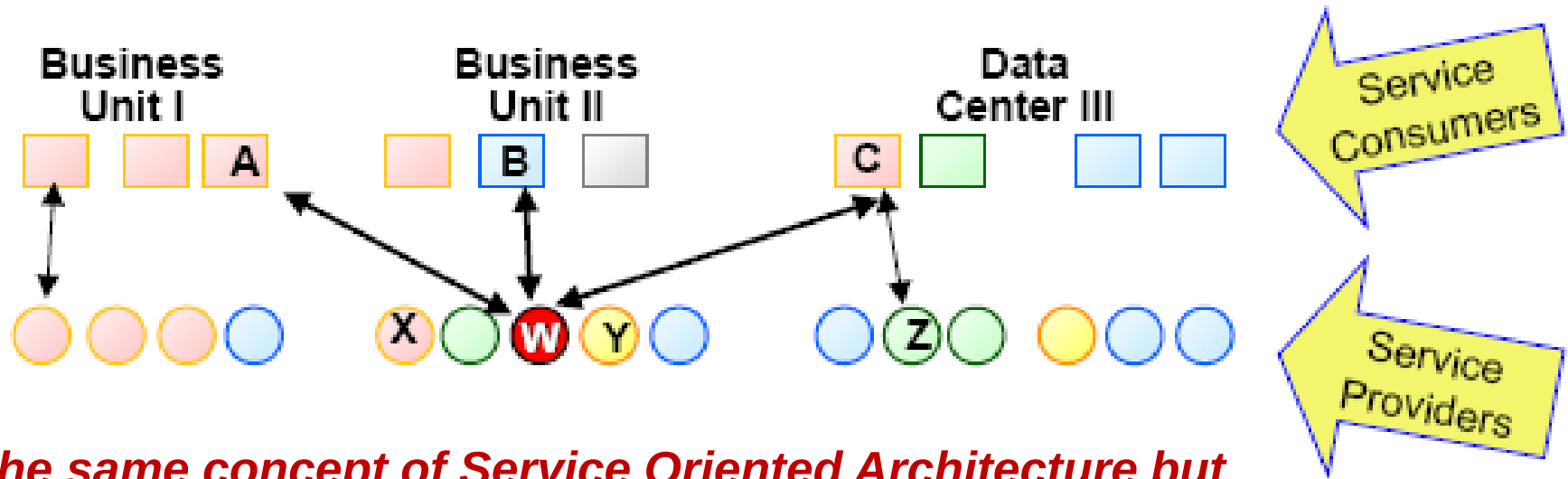
Flatter organization $\leftrightarrow$ Flatter production system



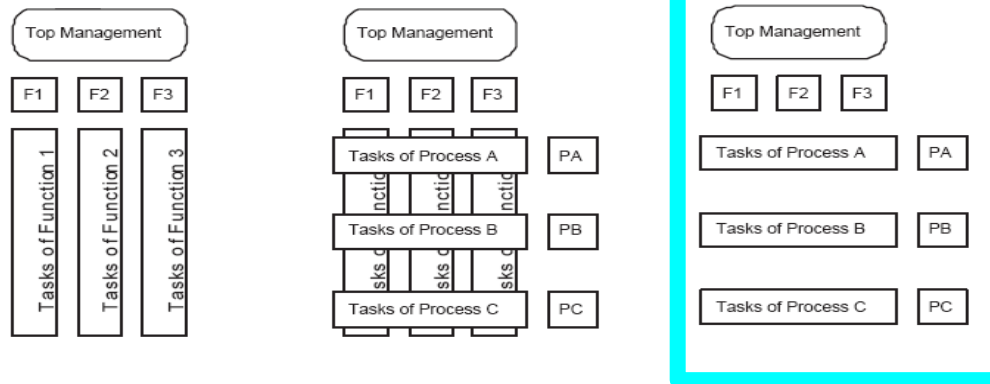
© Jacob Morgan (thefutureorganization.com)



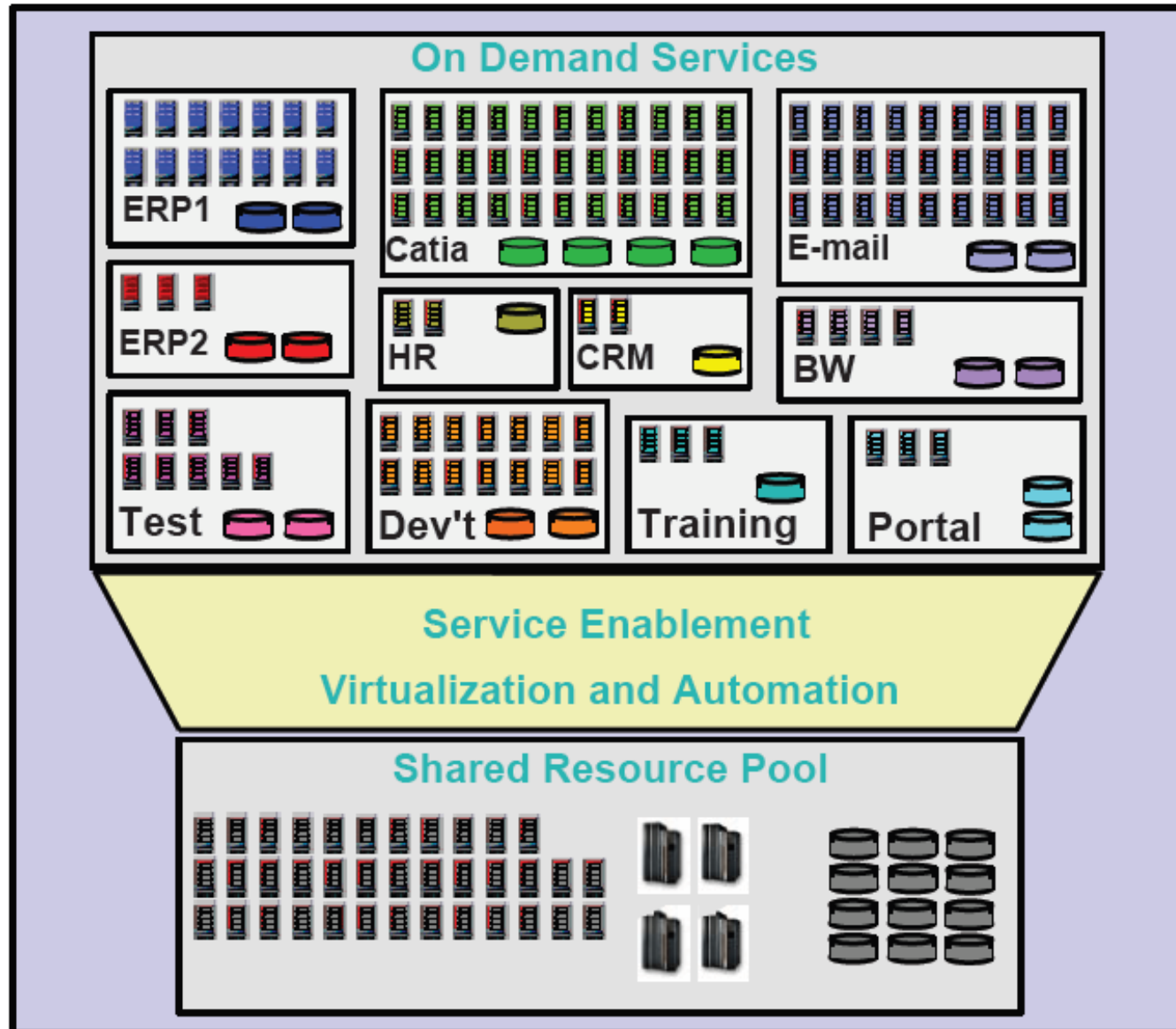
Then *Services* and *Micro-services*



The same concept of Service Oriented Architecture but finer grained, and based on asynchronous (event-based) lightweight protocols for communications



Modern infrastructure for services



Lines of Business View

Data Center View

Other “flatter” models

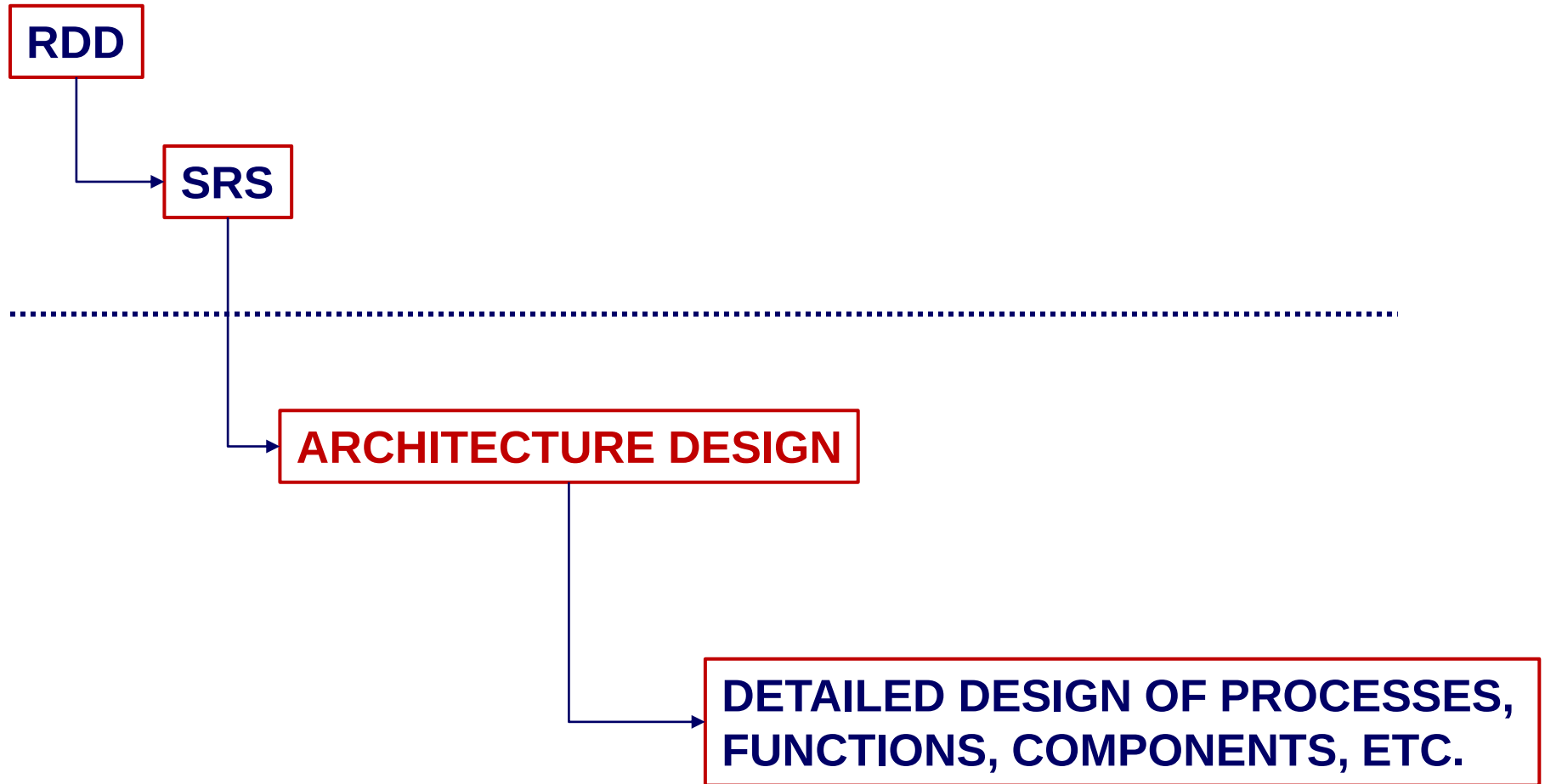
- **Agile**
- **DevOps**
- **DevSecOps**



Architecture design



Design process



Motivations for (adequate) design



Can be built by **one person**

Requires:

- minimal modeling
- simple process
- simple tools



Built by a **team**

Requires:

- modeling
- well-defined process
- power tools



Built by **several enterprises**

Requires:

- several modeling phases
- complex processes
- several power tools



Differences among architectures

- Scale
- Process
- Cost
- Schedule
- Skills and dimension of development teams
- Materials and technologies
- Stakeholders
- Risks

Even your architecture design effort and style must be appropriate to the complexity of the project: not more, not less



Architecture

- The architecture definition comes at early stage of the overall system design process
- It may be carried out in parallel with some specification activities
- It represents the link between requirement specification and the multi-level processes of design
- The architecture identify major system components, their communications and the overall control



Importance (1)

- Architecture serves as the blueprint for the system but also for the entire project at the level of:
 - Team structure
 - Documentation organization
 - Work breakdown structure
 - Software development model
 - Scheduling, planning, budgeting
 - Unit testing, integration



The power relationship with modern services

Validation

Testing

DESIGN

DEVELOPMENT

**OPERATION with
CONTINUOUS CHANGES**



Many terms – One goal

To produce quality code faster

- Agile
- DevOps
- Continuous Integration / Continuous Delivery (CI/CD)

Unfortunately, faster does not mean more resilient nor more secure



Main principles of Agile

The Development and Testing phases are integrated

- Individuals and interactions *over* processes and tools
- Working software *over* comprehensive documentation
- Customer collaboration *over* contract negotiation
- Responding to change *over* following a plan



The Agile manifesto (2001)

- 1) Our highest priority is to **satisfy the customer** through early and *continuous delivery* of valuable software
- 2) **Welcome changing requirements, even late in development.** Agile processes harness change for the customer's competitive advantage
- 3) **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale
- 4) **Business people and developers must work together daily throughout the project**
- 5) Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done
- 6) The most efficient and effective method of conveying information to and within a development team is face-to-face conversation



The Agile manifesto (2001)

- 7) **Working software is the primary measure of progress**
- 8) Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pass indefinitely
- 9) Continuous attention to **technical excellence** and **good design** enhances agility
- 10) **Simplicity** (the art of maximizing the amount of work not done) is essential
- 11) **The best architectures, requirements, and designs emerge from self-organizing teams**
- 12) At regular intervals, **the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly**



Problems with Agile

- **The Development and Testing phases are integrated but operation is not involved. Operation and development teams work in isolated environments**
- The operation team has no idea of the progress of the development team and has to prepare infrastructure and monitoring plan as per their own understanding
- Testing and Deployment activities are performed in an isolated manner after the design-build steps, and they take more time than actual project completion time
- Human production errors are deployed during manual coding
- Team members usually spend a large amount of time in deploying, testing, designing, and building the projects, but development and operation teams have different timelines and do not have proper synchronization that results in further delay



Real goal

We don't want to produce software faster

We want “go live” faster

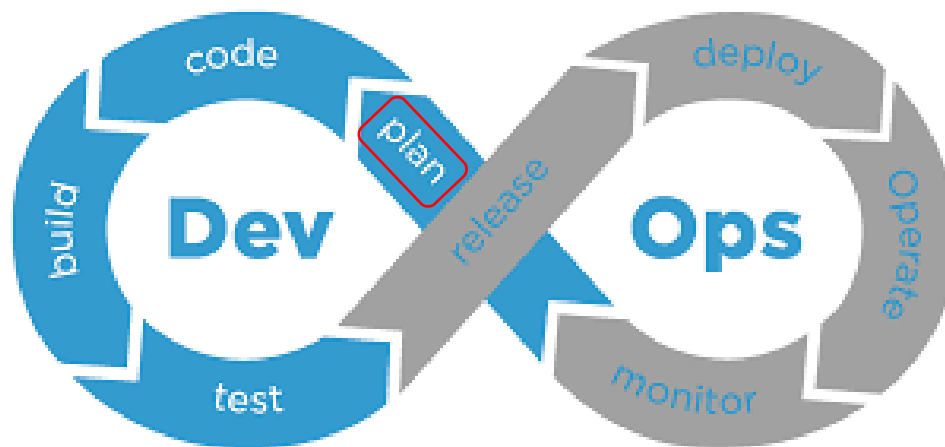


The rise of DevOps (~2009)

DevOps = Development + Operations

Goal: to increase the speed of delivering applications and services that work immediately

- It is not a platform. It is not a tool. It is a **culture** to promote development and operation process collaboratively
- It is a never-ending process of continual improvement



DevOps

- **DevOps extends the Agile principles beyond boundaries of the code to the **entire delivered services**** thus aligning system IT and development operations with better and improved communication
- It adopts *agile practices* in the context of a system-oriented approach
 - **Development, Testing and Operations are integrated**
 - The Operations team have a complete idea of the progress of development. Operations team and development team work together to develop a monitoring plan that caters to the business and IT needs



DevOps culture



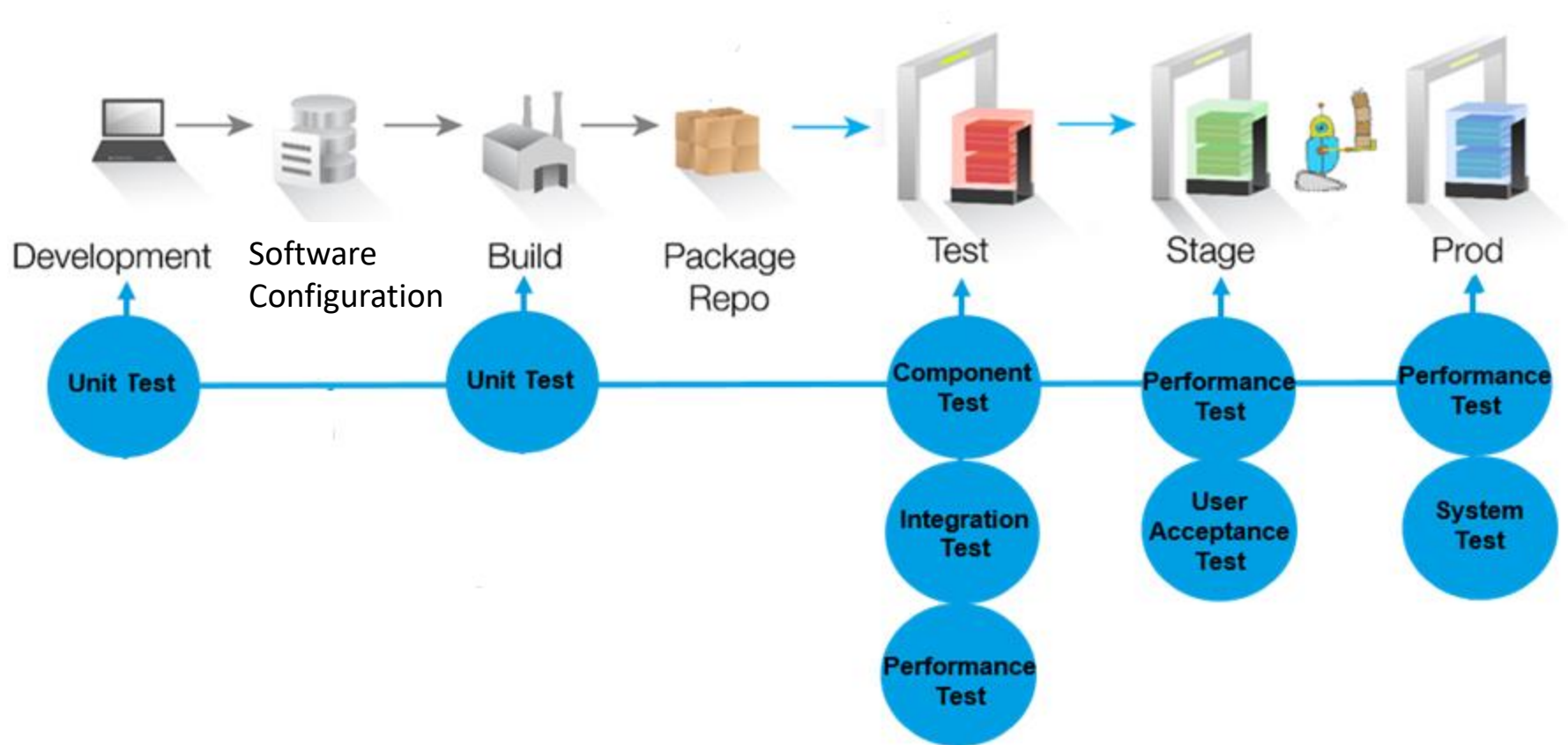
A group of experts is responsible for each stage of DevOps application development, including developers, testing, system administrators that work side by side

DevOps lifecycle

1. **Development:** The entire development process is broken down into small steps (*development cycles*). The speed of software development and delivery is increased
2. **Testing:** Testing tools are used during the development phase to speed up the overall testing process by quick identification of errors and fixing bugs
3. **Integration:** New functions are integrated with the prevailing code. Testing of new code takes place. Continuous integration and testing help in the continuous development process
4. **Deployment:** Continuous deployment of new functions and software is part of the DevOps lifecycle
5. **Monitoring:** Inappropriate system behavior is detected by continuous monitoring. Any bugs and anomalies are found and fixed in a hassle-free way



DevOps lifecycle



Effective service administration

- Code deployment time is reduced because of the faster testing process and speedy feedback mechanism
- Developers do not feel work pressure and can build new code in less time
- Operation management helps the organization to execute and organize the configuration plans, and they become able to manage their infrastructure in an effective way
- Containers and virtualization provide a simulated environment to increase software reliability and maintain uptime of the production environment
- Effective and improved server monitoring can provide immediate and proper feedback



DevOps: Possible roles and competences

Release Manager	Planning and software lifecycle management
DevOps Lead	Jenkins, Git, Nagios, Zabbix, bit bucket, Subversion (SVN), Chef, puppet, Ansible
DevOps Automation Expert	Git, SVN, Bitbucket, Jenkins
DevOps Testing Professional	Unit testing, build testing, Selenium, MySQL
DevOps Architect	C, C++, Java, Ruby, Python
DevOps System Engineer	Linux; AWS, GCP or Azure platforms

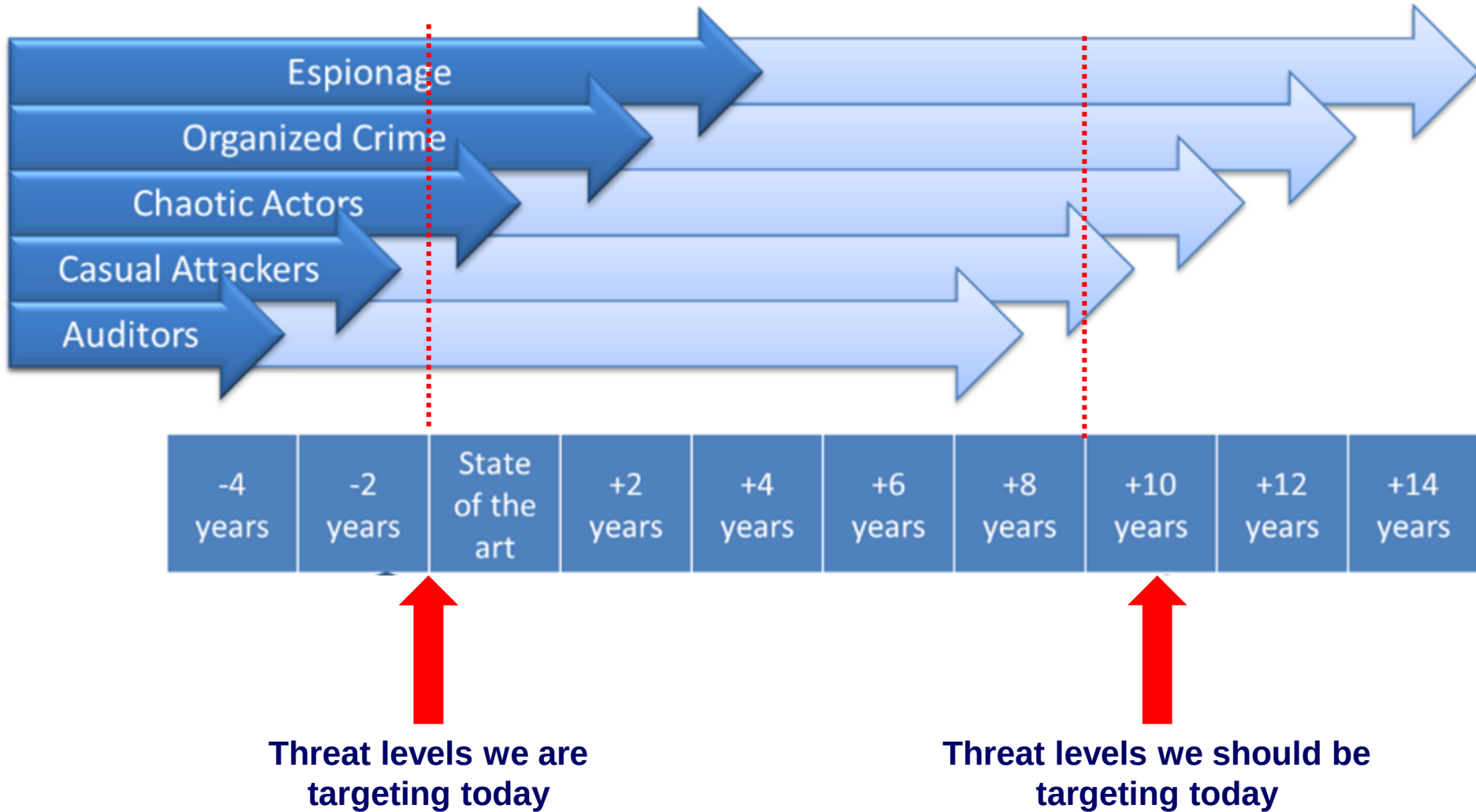


A further step towards a better (cyber) world

You don't want to produce software faster
You want “go live” faster with a reliable software



Software threats



Secure Development and Operations (*DevSecOps*)

- Approaches based on iterations over small units of work promote testing, fast feedback and room for experimentation. This is the best approach to build complex software systems and to maintain them
- Cybersecurity is seen as an obstacle for DevOps adoption
- Too often, cybersecurity is treated as an afterthought, only when some vulnerabilities are found

GOAL: To integrate security assurance in DevOps cycle without increasing (much) delay



The overall picture of DevSecOps

